Revision Game Cards — 2.1 Elements of Computational Thinking

Print and cut along the borders. ✂️ Loop cards: each player reads the bottom "Who has…?"; whoever holds the matching "I have…" reads it out, then their own clue — the chain visits every card and loops back to the start.

Loop cards (dominoes) — cut out & deal

1 / 16

I HAVE

Abstraction

WHO HAS…

the skill of breaking a problem into an ordered sequence of smaller sub-problems??

2 / 16

I HAVE

Decomposition

WHO HAS…

planning before coding: identifying inputs, outputs, preconditions and reusable parts??

3 / 16

I HAVE

Thinking ahead

WHO HAS…

a condition that must be true before a routine can run correctly??

4 / 16

I HAVE

Precondition

WHO HAS…

storing expensive or frequently used results in fast storage for reuse??

5 / 16

I HAVE

Caching

WHO HAS…

cached data that is now out of date because the source has changed??

6 / 16

I HAVE

Stale data

WHO HAS…

a self-contained, generalised module that can be used in many programs??

7 / 16

I HAVE

Reusable component

WHO HAS...

the skill of identifying where the flow of control branches on a condition??

8 / 16

I HAVE

Thinking logically

WHO HAS...

a place in a program where execution branches, e.g. an IF or CASE??

9 / 16

I HAVE

Decision point

WHO HAS...

the order in which statements execute, shaped by those branches??

10 / 16

I HAVE

Flow of control

WHO HAS...

the skill of spotting parts that can run at the same time??

11 / 16

I HAVE

Thinking concurrently

WHO HAS...

a fault where the result depends on the unpredictable timing of shared tasks??

12 / 16

I HAVE

Race condition

WHO HAS...

when tasks each wait for a resource the other holds, so none proceed??

13 / 16

I HAVE

Deadlock

WHO HAS...

multiple tasks running literally simultaneously on multiple cores??

14 / 16

I HAVE

Parallel processing

WHO HAS...

abstraction that removes detail from one specific thing to model that problem??

I HAVE

Representational abstraction

WHO HAS...

abstraction that groups many things by shared characteristics into a general model??

I HAVE

Generalisation (categorisation)

WHO HAS...

removing or hiding unnecessary detail so only relevant information remains??

Taboo cards — describe the term without the banned words

ABSTRACTION

Don't say:

- detail
- remove
- hide
- simplify

DECOMPOSITION

Don't say:

- break
- smaller
- sub-problem
- parts

PRECONDITION

Don't say:

- before
- true
- must
- runs

CACHING

Don't say:

- store
- fast
- reuse
- results

CONCURRENCY

Don't say:

- same time
- parallel
- together
- simultaneous

RACE CONDITION

Don't say:

- timing
- shared
- order
- unpredictable

REUSABLE COMPONENT

Don't say:

- module
- again
- self-contained
- library

DECISION POINT

Don't say:

- IF
- branch
- condition
- flow

"Apply it" cards — give a valid application of the named skill

APPLY: THINKING LOGICALLY
online chess game

APPLY: CACHING
weather app

APPLY: THINKING CONCURRENTLY
ride-booking app (e.g. taxis)

APPLY: THINKING AHEAD (INPUTS/OUTPUTS)
self-checkout till

APPLY: THINKING ABSTRACTLY
a road satnav

APPLY: THINKING PROCEDURALLY
school timetable generator

APPLY: THINKING AHEAD (PRECONDITION)
ATM / cash machine

APPLY: REUSABLE COMPONENTS
photo-editing software

APPLY: THINKING CONCURRENTLY (TRADE-OFF)
multiplayer quiz app

APPLY: THINKING LOGICALLY
e-commerce checkout

"Apply it" — model answers (teacher)

Scenario	Skill	Model application
online chess game	thinking logically	Give one decision point with an exact Boolean and both branches. IF move is legal -> TRUE: update the board and switch player; FALSE: reject the move and ask again.
weather app	caching	Explain one use of caching plus a trade-off. Cache the latest forecast so repeated opens load instantly and cut API calls; trade-off: it can go stale if the forecast updates, so refresh on a timer.
ride-booking app (e.g. taxis)	thinking concurrently	Name two tasks that can run at once and one trade-off. Track the car's GPS while the UI updates the map; trade-off: harder to debug and risk of inconsistent state needing synchronisation.
self-checkout till	thinking ahead (inputs/outputs)	State two inputs and one output. Inputs - scanned barcode, payment; Output - receipt / payment accepted message.
a road satnav	thinking abstractly	Name one detail kept and one removed, with reason. Keep road connections and distances; remove the colour of the buildings - irrelevant to finding the shortest route.
school timetable generator	thinking procedurally	Name three ordered sub-procedures. 1) Read teacher/room availability; 2) Assign classes to free slots; 3) Output the finished timetable.
ATM / cash machine	thinking ahead (precondition)	State a precondition that must hold before dispensing cash. The account balance must be at least the requested amount (and the PIN already verified).
photo-editing software	reusable components	Give one reusable component and a benefit. A blur filter module reused by many tools - faster development and fewer bugs as it is already tested.
multiplayer quiz app	thinking concurrently (trade-off)	Identify a risk of letting players answer simultaneously. A race condition when two answers update the shared scoreboard at once - needs locking/synchronisation; not automatically faster.
e-commerce checkout	thinking logically	Give one decision point with an exact Boolean and both branches. IF stockLevel >= quantityOrdered -> TRUE: confirm the order and reduce stock; FALSE: show out of stock.

Quick-fire — clue & answer (self-test / quiz-quiz-trade)

#	Clue	Answer
1	I removed the room's carpet colour from my model because it doesn't affect bookings.	Thinking abstractly
2	I worked out the inputs, outputs and what must be true first, before writing any code.	Thinking ahead
3	I split 'make a booking' into check, store and confirm steps in a fixed order.	Thinking procedurally
4	I wrote IF seatsRequested <= seatsAvailable and planned both branches.	Thinking logically
5	I spotted that logging and rendering the screen have no data dependency, so they can overlap.	Thinking concurrently
6	The list must be sorted before this binary search will work.	Precondition
7	I stored the frequently requested results in fast memory to avoid recomputing them.	Caching
8	My cached price list is wrong because the database changed but the cache didn't.	Stale data
9	Two students booked the same slot at the same instant and the result depended on timing.	Race condition
10	I grouped savings and current accounts into one general Account model.	Generalisation / categorisation (abstraction)
11	Each task is waiting for a resource the other is holding, so nothing moves.	Deadlock
12	I made one tested, self-contained module and used it across three programs.	Reusable component